Code Explanation

File Generator Module Explanation

The provided code is a Python module named `file_generator.py` that generates 10K flat files with proper naming and partitioning using Apache Spark. It contains two main functions: `calculate_filename` and `write_10k_files`.

Overview

This module is designed to take a Spark DataFrame and configuration parameters as input, calculate the file name for each record based on the configuration, and write the data to 10K flat files using transaction control.

Step 1: Calculating the Base Filename

The `calculate_filename` function starts by calculating the base filename based on the current date and configuration parameters. It takes into account the `file_prefix`, `batch_id`, and `run_id` to construct the base filename.

```
current_date = datetime.now().strftime("%Y%m%d")
base_filename = f"{config['file_prefix']}_{current_date}"
if config['batch_id']:
    base_filename += f"_{config['batch_id']}"
if config['run_id']:
    base_filename += f"_{config['run_id']}"
```

Step 2: Adding Row Number and File Sequence Number

The function then adds a row number column to the input DataFrame using `monotonically_increasing_id()`. It calculates the file sequence number for each row by dividing the row number by `max_rows_per_file` and casting the result to an integer.

```
df_with_row_num = df.withColumn("row_num", monotonically_increasing_id())
df_with_file_seq = df_with_row_num.withColumn(
    "file_seq",
    (col("row_num") / config['max_rows_per_file']).cast("int") + 1
)
```

Step 3: Creating the Full File Path

The function creates the full file path for each row by concatenating the output directory, base filename, file sequence number, and file extension.

```
df_with_filename = df_with_file_seq.withColumn(
    "File_Path_Name",
    concat(
        lit(os.path.join(config['output_dir'], base_filename)),
        lit("_"),
        col("file_seq").cast("string"),
        lit("."),
        lit(config['file_ext'])
    )
```

```
)
```

Step 4: Dropping Temporary Columns
The function drops the temporary columns (`row_num` and `file_seq`) from the DataFrame.

```
result_df = df_with_filename.drop("row_num", "file_seq")
```

Step 5: Writing Data to 10K Flat Files
The `write_10k_files` function writes the data to 10K flat files using transaction control. It first ensures that the output directory exists.

```
os.makedirs(config['output_dir'], exist_ok=True)
```

Step 6: Repartitioning the DataFrame
The function repartitions the DataFrame based on the `File_Path_Name` column to ensure that each file is written separately.

```
partitioned_df = df.repartition("File_Path_Name")
```

Step 7: Writing the Files
The function writes the files using the `csv` format with a custom delimiter (`|`) to simulate a flat file.

```
partitioned_df.write
    .format("csv")
    .option("header", "false")
    .option("delimiter", "|")
    .option("encoding", "UTF-8")
    .option("quote", "u0000")
    .mode("overwrite")
    .partitionBy("File_Path_Name")
    .save(f"{config['output_dir']}/temp")
```

Step 8: Renaming the Files
The function renames the files to their actual names by moving them from the temporary directory to their final locations.

```
part_files = glob.glob(f"{config['output_dir']}/temp/File_Path_Name=
*/*.csv")
for part_file in part_files:
    dir_name = os.path.dirname(part_file)
    file_path_name = dir_name.split("File_Path_Name=")[1].replace("%
2F", "/")
    target_dir = os.path.dirname(file_path_name)
    os.makedirs(target_dir, exist_ok=True)
    subprocess.run(["mv", part_file, file_path_name])
```

Step 9: Cleaning Up
The function cleans up the temporary directory.

```
subprocess.run(["rm", "-rf", f"{config['output_dir']}/temp"])
```